Some Theory

CSCI 1101

2024-03-29

Why Learn Theory?

Each of the types of data we have encountered has a corresponding mathematical theory.

Understanding that theory can give you deeper insight into your programs and make you a better programmer.

Theory of Booleans

Two boolean expressions p and q are **logically equivalent**, written "$p \Leftrightarrow q$" just in case they have the same value as each other for all possible parameters.

For example, we can test whether the following functions are equivalent by testing four cases:

```
def be1(p , q) :
    return not (p or (not p and q))

def be2(p , q) :
    return not p and (p or not q)
```

With just four cases to check this is not so bad, but as the number of parameters increases it becomes infeasible.

Instead, we can reason about the logic of `True` ($\top$), `False` ($\bot$), `and` ($\wedge$), `or` ($\vee$), and `not` ($\neg$) directly using **Boolean algebra**.

Boolean Algebra

Both $\wedge$ and $\vee$ are **associative**:

$$(p \wedge q) \wedge r \;\Leftrightarrow\; p \wedge (q \wedge r) \qquad \text{and} \qquad (p \vee q) \vee r \;\Leftrightarrow\; p \vee (q \vee r)$$

$\wedge$ and $\vee$ have **neutral elements** $\top$ and $\bot$ respectively:

$$\top \wedge p \;\Leftrightarrow\; p \;\Leftrightarrow\; p \wedge \top \qquad \text{and} \qquad \bot \vee p \;\Leftrightarrow\; p \;\Leftrightarrow\; p \vee \bot$$

The neutral element for each is an **absorbing element** for the other:

$$\bot \wedge p \;\Leftrightarrow\; \bot \;\Leftrightarrow\; p \wedge \bot \qquad \text{and} \qquad \top \vee p \;\Leftrightarrow\; \top \;\Leftrightarrow\; p \vee \top$$

This lets us $\wedge$ or $\vee$ together any *list* of booleans using *short-circuit evaluation*.

In Python we do this using the functions:

```
all , any  :  (list bool) --> bool
```

Boolean Algebra (ctd.)

Both $\wedge$ and $\vee$ are **idempotent**:

$$p \wedge p \;\Leftrightarrow\; p \;\Leftrightarrow\; p \vee p$$

Both $\wedge$ and $\vee$ are **commutative**:

$$p \wedge q \;\Leftrightarrow\; q \wedge p \qquad \text{and} \qquad p \vee q \;\Leftrightarrow\; q \vee p$$

Since multiplicity and order don't matter we can $\wedge$ or $\vee$ together any *set* of booleans.

In Python the `all` and `any` functions are overloaded to act on sets as well:

```
all , any  :  (set bool) --> bool
```

Distributive Laws

$\wedge$ **distributes** over $\vee$:

$$p \wedge (q \vee r) \;\Leftrightarrow\; (p \wedge q) \vee (p \wedge r)$$

$\vee$ **distributes** over $\wedge$ as well:

$$p \vee (q \wedge r) \;\Leftrightarrow\; (p \vee q) \wedge (p \vee r)$$

Because of commutativity, we can distribute on the right as well as on the left.

Negation Laws

Negation is an **involution**:

$$\neg(\neg p) \;\Leftrightarrow\; p$$

Combining any boolean with its own negation gives us the absorbing element:

$$p \wedge \neg p \;\Leftrightarrow\; \bot \qquad \text{and} \qquad p \vee \neg p \;\Leftrightarrow\; \top$$

$\neg$ turns $\top$ and $\bot$ into each other:

$$\neg\top \;\Leftrightarrow\; \bot \qquad \text{and} \qquad \neg\bot \;\Leftrightarrow\; \top$$

$\neg$ distributes over $\wedge$ and $\vee$ according to the **De Morgan laws**:

$$\neg(p \wedge q) \;\Leftrightarrow\; \neg p \vee \neg q \qquad \text{and} \qquad \neg(p \vee q) \;\Leftrightarrow\; \neg p \wedge \neg q$$

Challenge: Boolean Algebra

Use the laws of boolean algebra to prove that the following two expressions are logically equivalent:

$$\text{be1} := \neg(p \vee (\neg p \wedge q))$$

$$\text{be2} := \neg p \wedge (p \vee \neg q)$$

Theory of Sets

The algebra of sets is closely related to the algebra of booleans.

Recall:

$$x \in \mathrm{A} \cap \mathrm{B} \quad \Leftrightarrow \quad x \in \mathrm{A} \wedge x \in \mathrm{B}$$
$$x \in \mathrm{A} \cup \mathrm{B} \quad \Leftrightarrow \quad x \in \mathrm{A} \vee x \in \mathrm{B}$$

From this we can immediately infer that $\cap$ and $\cup$ are:

associative: $(\mathrm{A} \cap \mathrm{B}) \cap \mathrm{C} = \mathrm{A} \cap (\mathrm{B} \cap \mathrm{C})$ and $(\mathrm{A} \cup \mathrm{B}) \cup \mathrm{C} = \mathrm{A} \cup (\mathrm{B} \cup \mathrm{C})$,

idempotent: $\mathrm{A} \cap \mathrm{A} = \mathrm{A} = \mathrm{A} \cup \mathrm{A}$,

commutative: $\mathrm{A} \cap \mathrm{B} = \mathrm{B} \cap \mathrm{A}$ and $\mathrm{A} \cup \mathrm{B} = \mathrm{B} \cup \mathrm{A}$,

distributive: $\mathrm{A} \cap (\mathrm{B} \cup \mathrm{C}) = (\mathrm{A} \cap \mathrm{B}) \cup (\mathrm{A} \cap \mathrm{C})$ and $\mathrm{A} \cup (\mathrm{B} \cap \mathrm{C}) = (\mathrm{A} \cup \mathrm{B}) \cap (\mathrm{A} \cup \mathrm{C})$.

Example: Distributivity for Sets

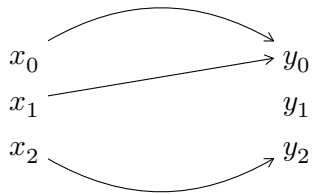
$$
\begin{aligned}
& x \in \mathrm{A} \cap (\mathrm{B} \cup \mathrm{C}) \\
\Leftrightarrow\ & [\text{definition of set intersection}] \\
& x \in \mathrm{A} \ \wedge\ x \in \mathrm{B} \cup \mathrm{C} \\
\Leftrightarrow\ & [\text{definition of set union}] \\
& x \in \mathrm{A} \ \wedge\ (x \in \mathrm{B} \ \vee\ x \in \mathrm{C}) \\
\Leftrightarrow\ & [\text{distribution for booleans}] \\
& (x \in \mathrm{A} \ \wedge\ x \in \mathrm{B}) \ \vee\ (x \in \mathrm{A} \ \wedge\ x \in \mathrm{C}) \\
\Leftrightarrow\ & [\text{definition of set intersection}] \\
& x \in \mathrm{A} \cap \mathrm{B} \ \vee\ x \in \mathrm{A} \cap \mathrm{C} \\
\Leftrightarrow\ & [\text{definition of set union}] \\
& x \in (\mathrm{A} \cap \mathrm{B}) \cup (\mathrm{A} \cap \mathrm{C})
\end{aligned}
$$

Functions

A mathematical **function** is a relationship between two sets, called its **domain** and **codomain**, with the following property:

each element of the domain is related to *exactly one* element of the codomain.

For example:



We write "$f : \mathrm{A} \to \mathrm{B}$" to mean f is a function with domain A and codomain B.

And write "$f(x) = y$" to mean y is the unique element related to x by f.

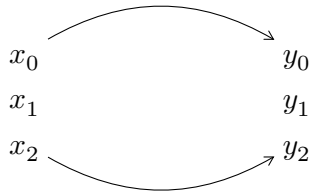
Python functions may or may not be mathematical functions.

Partial Functions

A **partial function** is more general than a function:

each element of the domain is related to *at most one* element of the codomain.

For example:



Python dictionaries encode partial functions: each possible key is associated with at most one value.

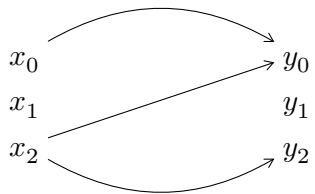
We can look up the value associated with a key *if* that key is in the dictionary.

Relations

A mathematical **relation** is more general than a partial function:

each element of the domain can be related to *any number of* elements of the codomain.

For example:



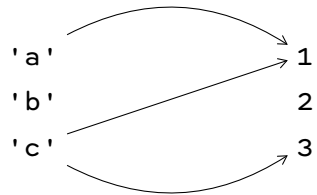
We write "$\mathrm{R} : \mathrm{A} \nrightarrow \mathrm{B}$" to mean R is a relation with domain A and codomain B.

And write "$x\mathrm{R}y$" to mean $x \in \mathrm{A}$ and $y \in \mathrm{B}$ are related by R.

Relations in Python

In Python we can encode a relation $\mathrm{R} : \mathrm{A} \nrightarrow \mathrm{B}$ using a dictionary `R : dict (A , set B)`.

For example, let R be the following relation:



We can encode this relation as the following `dict (str , set int)`:

```
R = {'a':{1} , 'c':{1,3}}
```

Activity: Relations in Python

Write a function `related : (dict (a , set b) , a , b) --> bool` so that `related (rel , x , y)` determines whether `x` and `y` are related by `rel`.

For example:

```
related(R , 'a' , 1)
True
related(R , 'a' , 2)
False
related(R , 'c' , 1)
True
related(R , 'c' , 3)
True
```

To Do

Finish *Homework 8: Sets and Dictionaries* on CodeRunner by 11:00AM next Thursday.

Finish *Project 3 - Automated Grader (Part 1)* on CodeRunner by midnight next Friday.